CS 216: Bitcoin Transaction Lab

Team Name	HighOnByte
Course	CS 216
Assignment	2nd Programming Assignment
Deadline	10th March 2026

Team Members

Name	Roll Number
Aastha	240004001
Bhavika Jaiswal	240001017
Sanskriti Jain	240001064
Suhani	240001077

GitHub Repository:
https://github.com/Sanskriti-hello/bitcoin-transaction-lab

Contents

1 Theoretical Background **2**
- 1.1 Bitcoin Script . . . 2
- 1.2 P2PKH – Pay to Public Key Hash (Legacy) . . . 2
- 1.3 P2SH-P2WPKH – SegWit wrapped in P2SH . . . 2

2 Part 1: Legacy P2PKH Transactions **4**
- 2.1 Bitcoin Core Configuration . . . 4
- 2.2 Generated Legacy Addresses . . . 4
- 2.3 Transaction Workflow: A to B to C . . . 4
- 2.4 Decoded Scripts – Transaction A to B . . . 4
- 2.5 Decoded Scripts – Transaction B to C . . . 5
- 2.6 Transaction Sizes – P2PKH . . . 5

3 Part 1: btcdeb Script Execution (P2PKH) **6**
- 3.1 Command Used . . . 6
- 3.2 Initial Script Load – 8 Opcodes . . . 6
- 3.3 Step-by-Step Execution . . . 6
- 3.4 Final Result . . . 8

4 Part 2: P2SH-P2WPKH SegWit Transactions **9**
- 4.1 Generated SegWit Addresses . . . 9
- 4.2 Transaction Workflow: A' to B' to C' . . . 9
- 4.3 Decoded Scripts – Transaction A' to B' . . . 9
- 4.4 Decoded Scripts – Transaction B' to C' . . . 9
- 4.5 Transaction Sizes – SegWit . . . 10

5 Part 2: btcdeb Script Execution (SegWit) **11**
- 5.1 Command Used . . . 11
- 5.2 Initial Load – Witness Stack Size 2 . . . 11
- 5.3 Step-by-Step Execution . . . 11
- 5.4 Final Result . . . 12

6 Part 3: Comparative Analysis **13**
- 6.1 Transaction Size Comparison . . . 13
- 6.2 Script Structure Comparison . . . 13
- 6.3 Weight Calculation – Worked Example . . . 13
- 6.4 Why SegWit Transactions Are Smaller . . . 13

7 Conclusion **15**

1 Theoretical Background

1.1 Bitcoin Script

Bitcoin Script is a simple, stack-based programming language used to define the conditions under which Bitcoin can be spent. It has no loops and is not Turing-complete, making it simple and safe to execute. Transaction validation works by combining two scripts:

- **scriptSig** (unlocking script) – provided by the spender in the input
- **scriptPubKey** (locking script) – stored in the UTXO being spent

They are concatenated and executed together. The transaction is valid only if the final stack value is `TRUE (0x01)`.

1.2 P2PKH – Pay to Public Key Hash (Legacy)

P2PKH is the classic Bitcoin address format. In regtest mode, addresses start with `m` or `n`.

Address derivation:

```
Private Key -> Public Key -> SHA256 -> RIPEMD160 -> Bitcoin Address
```

Challenge Script (scriptPubKey) – 25 bytes

```
OP_DUP OP_HASH160 <pubKeyHash_20bytes> OP_EQUALVERIFY OP_CHECKSIG
hex prefix: 76a914...88ac
```

Response Script (scriptSig) – 106 bytes

```
<DER_signature ~71bytes> <compressed_pubkey_33bytes>
```

Stack Execution Trace

Step	Operation	Stack State
0	Push signature (from scriptSig)	[sig]
1	Push pubkey (from scriptSig)	[sig, pubKey]
2	OP_DUP	[sig, pubKey, pubKey]
3	OP_HASH160	[sig, pubKey, hash160(pubKey)]
4	Push expected hash (from scriptPubKey)	[sig, pubKey, computed, expected]
5	OP_EQUALVERIFY – must match	[sig, pubKey]
6	OP_CHECKSIG – verify ECDSA sig	[TRUE / 0x01]

1.3 P2SH-P2WPKH – SegWit wrapped in P2SH

SegWit (BIP 141) separates signature data into a **witness** field. In P2SH-P2WPKH, a native SegWit script is wrapped inside a P2SH address for backward compatibility. Addresses start with `2` in regtest.

scriptPubKey (on-chain) – 23 bytes

```
OP_HASH160 <hash160(redeemScript)> OP_EQUAL
hex prefix: a914...87
```

redeemScript (revealed when spending) – 22 bytes

```
OP_0 <hash160(publicKey)>
hex: 0014<20bytes>
```

scriptSig – 23 bytes only

```
<serialized redeemScript>   <- only the redeem script, NOT the signature
```

Witness (segregated, weight discounted)

```
[0]: <DER_signature ~71bytes>
[1]: <compressed_pubkey 33bytes>
```

Weight Calculation

```
weight = (non-witness bytes x 4) + (witness bytes x 1)
vbytes = ceil(weight / 4)
Witness bytes get 75% discount compared to regular bytes.
```

2 Part 1: Legacy P2PKH Transactions

2.1 Bitcoin Core Configuration

`bitcoind` was run in regtest mode with the following `bitcoin.conf`:

```
regtest=1
server=1
txindex=1

[regtest]
rpcuser=student
rpcpassword=cs216bitcoin
rpcport=18443
rpcbind=127.0.0.1
rpcallowip=127.0.0.1
paytxfee=0.0001
fallbackfee=0.0002
mintxfee=0.00001
txconfirmtarget=6
```

2.2 Generated Legacy Addresses

Three legacy (P2PKH) addresses were generated using `getnewaddress "label" "legacy"`:

Label	Address
Address A	`msHDz1o3mFxQjKBw7CAxrLW1crF2TTpr7b`
Address B	`mxnRfURkaEPpGdmf6EqkmgGg9ggJGJU5cN`
Address C	`mtqvSHwe7fC6Eux36REkK9P7iDHk8H3Gjo`

2.3 Transaction Workflow: A to B to C

Address A was funded with 1.0 BTC. The UTXO created at A was used as input to transaction A to B. The UTXO at B was then used as input to B to C, forming a verified UTXO chain.

Funding TXID (coins sent to A)

```
899317b6f6cf0d1412e8c04aaabaa1730747e399b5c937e54b3a1876adbb1941
```

TXID: Transaction A to B

```
105e2c9e37135a3ab0404843b6df6452f5368bd40f64500ea93b8feb050a2e1a
```

TXID: Transaction B to C

```
160f61723c69bb41cd3a1bdace88b556760cca76189d35c28b68deb48b34cad3
```

Chain verification: The UTXO at address B was confirmed to have txid equal to the A to B transaction, verified programmatically using `assert utxo_b["txid"] == txid_ab`.

2.4 Decoded Scripts – Transaction A to B

Challenge Script (scriptPubKey) for Address B

```
type: pubkeyhash
asm:  OP_DUP OP_HASH160 bd67397892887d1af275e31173a53cd8a376107d
      OP_EQUALVERIFY OP_CHECKSIG
hex:  76a914bd67397892887d1af275e31173a53cd8a376107d88ac
size: 25 bytes
```

This script **locks** the output to address B. To spend it, the spender must provide a public key whose hash160 equals `bd67397892887d1af275e31173a53cd8a376107d` AND a valid ECDSA signature.

2.5 Decoded Scripts – Transaction B to C

Challenge Script (scriptPubKey) for Address C

```
type: pubkeyhash
asm:  OP_DUP OP_HASH160 922ffcce68bcb263789f5092e105ee8748cfd4a4
      OP_EQUALVERIFY OP_CHECKSIG
hex:  76a914922ffcce68bcb263789f5092e105ee8748cfd4a488ac
size: 25 bytes
```

Response Script (scriptSig) – unlocking B's UTXO

```
asm:
  304402200bde2b26b9eee139286f7bdd9011911436d8f8786a90128470938923037099 0
  502204709557cb37be9ac684baafea22a29af3d9bc981ae2bf16c3ceecd5b22acf551
  [ALL] 031888c85eeafd069fe78adc5dc90058090e813e8df215ce87af933015ae333005

hex:
  47304402200bde2b26b9eee139286f7bdd9011911436d8f8786a9012847093892303709
  90502204709557cb37be9ac684baafea22a29af3d9bc981ae2bf16c3ceecd5b22acf551
  0121031888c85eeafd069fe78adc5dc90058090e813e8df215ce87af933015ae333005

size: 106 bytes
  - DER signature: ~71 bytes (includes SIGHASH_ALL byte)
  - Compressed public key: 33 bytes
  - Push data opcodes: 2 bytes
```

Challenge-Response Match Verification

Challenge requires	Response provides
hash160(pubKey) == bd67397892...	pubKey = 031888c85e... whose hash160 = bd67397892...
valid ECDSA signature	sig = 304402200bde2b... verified against pubKey

2.6 Transaction Sizes – P2PKH

Transaction	Size (bytes)	vSize (vbytes)	Weight (WU)
A to B	191	191	764
B to C	191	191	764

For P2PKH: size = vsize = weight/4 because there is no witness data. All bytes counted at weight x4.

3 Part 1: btcdeb Script Execution (P2PKH)

3.1 Command Used

```
btcdeb --tx=<signed_hex_B_to_C> --txin=<signed_hex_A_to_B>
```

btcdeb loads the spending transaction (B to C) and the UTXO being spent (A to B output), reconstructs the full script execution, and steps through each opcode showing stack state.

3.2 Initial Script Load – 8 Opcodes

```
(venv) sunny@DESKTOP-FVSQ52O:~/bitcoin_lab$ btcdeb --tx=02000000014dbbd24563ff8a8c620136e9ac0f4f1a61c577195c35a44cb1baac
193b2c6663000000006a4730440220074610 5d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e3661cc9d4b07022060990535eb51d164db2d5c
68fd8528a0fc0e3ffa8cbb811a7a4527e572a0aa11012103ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959fdffffff
01e092f505000000001976a914b74dc52aaf7aeae49016fee8883238f63d2b0de988ac00000000 --txin=020000000126439c7e843967ede468c556
dee3bf298060b0ecd86340929445485357a612be000000006a473044022001caa2612a2a401223ec472b66f63ddcd713eecf21a820866fcc8f1fd726
3712022065aea4c8f86c84f6742a973f5cbbeac26e4a25f0d7e4441957c302a43a004b100121024bd1a0d388ba2f435940601d9a23356cd2ab157fc3
304dd76aeff20b9e6201b8fdffffff01f0b9f505000000001976a914f353305a8256b3a735a3c2913405fb74126d8ef188ac00000000
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
input tx index = 0; tx input vout = 0; value = 99990000
got witness stack of size 0
8 op script loaded. type `help` for usage information
script                                                              |  stack
--------------------------------------------------------------------+--------
304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e366... |
03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959 |
<<< scriptPubKey >>>                                                |
OP_DUP                                                              |
OP_HASH160                                                          |
f353305a8256b3a735a3c2913405fb74126d8ef1                            |
OP_EQUALVERIFY                                                      |
OP_CHECKSIG                                                         |
#0000 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e3661cc9d4b07022060990535eb51d164db2d5c68fd8528a0fc0e3f
fa8cbb811a7a4527e572a0aa1101
btcdeb>
```

Figure 1: btcdeb: Initial state – 8 op script loaded, witness stack size 0 (P2PKH/non-segwit). Both signature and pubkey visible in script column, scriptPubKey opcodes listed below.

3.3 Step-by-Step Execution

```
btcdeb> step
                <> PUSH stack 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e3661cc9d4b07022060990535eb51d164db2d5c68fd8528a0fc0e3ffa8cbb811a7a4527e572a0aa1101
script                                                              |                                                                  stack
--------------------------------------------------------------------+--------------------------------------------------------------------
03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959 | 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e366...
<<< scriptPubKey >>>                                                |
OP_DUP                                                              |
OP_HASH160                                                          |
f353305a8256b3a735a3c2913405fb74126d8ef1                            |
OP_EQUALVERIFY                                                      |
OP_CHECKSIG                                                         |
#0001 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
btcdeb>
```

Figure 2: Step 0: Signature pushed onto stack from scriptSig. Stack: [sig]

```
btcdeb> step
                <> PUSH stack 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
script                                                              |                                                                  stack
--------------------------------------------------------------------+--------------------------------------------------------------------
<<< scriptPubKey >>>                                                | 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
OP_DUP                                                              | 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e366...
OP_HASH160                                                          |
f353305a8256b3a735a3c2913405fb74126d8ef1                            |
OP_EQUALVERIFY                                                      |
OP_CHECKSIG                                                         |
<<< scriptPubKey >>>
btcdeb>
```

Figure 3: Step 1: Public key `03ac35ad9e45a8b806451de1...` pushed. Stack: [pubKey, sig]

```
btcdeb> step
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------------
OP_DUP                                                          | 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
OP_HASH160                                                      | 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e366...
f353305a8256b3a735a3c2913405fb74126d8ef1                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
#0003 OP_DUP
btcdeb>
```

Figure 4: Step 2 (OP_DUP): Top of stack duplicated. Stack: [pubKey, pubKey, sig]

```
btcdeb> step
                <> PUSH stack 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------------
OP_HASH160                                                      | 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
f353305a8256b3a735a3c2913405fb74126d8ef1                        | 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
OP_EQUALVERIFY                                                  | 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e366...
OP_CHECKSIG                                                     |
#0004 OP_HASH160
btcdeb>
```

Figure 5: Step 3 (OP_HASH160): Top pubKey hashed. Stack: [hash160(pubKey), pubKey, sig]

```
btcdeb> step
                <> POP  stack
                <> PUSH stack f353305a8256b3a735a3c2913405fb74126d8ef1
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------------
f353305a8256b3a735a3c2913405fb74126d8ef1                        |                         f353305a8256b3a735a3c2913405fb74126d8ef1
OP_EQUALVERIFY                                                  | 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
OP_CHECKSIG                                                     | 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e366...
#0005 f353305a8256b3a735a3c2913405fb74126d8ef1
btcdeb>
```

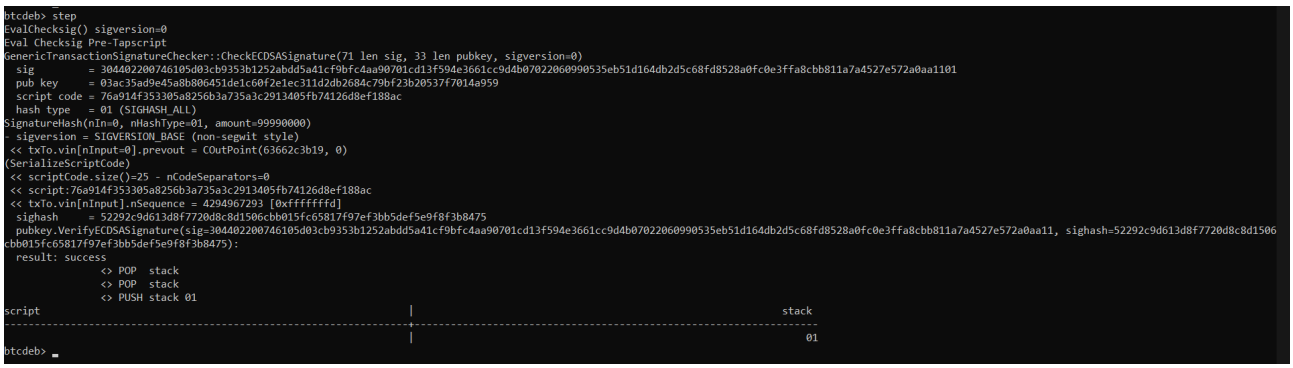
Figure 6: Step 4: Expected hash `f353305a8256b3a735a3c2913405fb74126d8ef1` pushed. Stack: [expected, computed, pubKey, sig]

```
btcdeb> step
                <> PUSH stack f353305a8256b3a735a3c2913405fb74126d8ef1
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------------
OP_EQUALVERIFY                                                  |                         f353305a8256b3a735a3c2913405fb74126d8ef1
OP_CHECKSIG                                                     |                         f353305a8256b3a735a3c2913405fb74126d8ef1
                                                                | 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
                                                                | 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e366...
#0006 OP_EQUALVERIFY
btcdeb>
```

Figure 7: Step 5 (OP_EQUALVERIFY): Both hashes compared. They match – both popped. Stack: [pubKey, sig]

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------------
OP_CHECKSIG                                                     | 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
                                                                | 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e366...
#0007 OP_CHECKSIG
btcdeb>
```

Figure 8: Step 6: OP_CHECKSIG about to execute – pubKey and sig on stack ready for ECDSA verification



```
btcdeb> step
EvalChecksig() sigversion=0
Eval Checksig Pre-Tapscript
GenericTransactionSignatureChecker::CheckECDSASignature(71 len sig, 33 len pubkey, sigversion=0)
  sig         = 304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e3661cc9d4b07022060990535eb51d164db2d5c68fd8528a0fc0e3ffa8cbb811a7a4527e572a0aa1101
  pub key     = 03ac35ad9e45a8b806451de1c60f2e1ec311d2db2684c79bf23b20537f7014a959
  script code = 76a914f353305a8256b3a735a3c2913405fb74126d8ef188ac
  hash type   = 01 (SIGHASH_ALL)
SignatureHash(nIn=0, nHashType=01, amount=99990000)
- sigversion = SIGVERSION_BASE (non-segwit style)
 << txTo.vin[nInput=0].prevout = COutPoint(63662c3b19, 0)
(SerializeScriptCode)
 << scriptCode.size()=25 - nCodeSeparators=0
 << script:76a914f353305a8256b3a735a3c2913405fb74126d8ef188ac
 << txTo.vin[nInput].nSequence = 4294967293 [0xfffffffd]
  sighash     = 52292c9d613d8f7720d8c8d1506cbb015fc65817f97ef3bb5def5e9f8f3b8475
  pubkey.VerifyECDSASignature(sig=304402200746105d03cb9353b1252abdd5a41cf9bfc4aa90701cd13f594e3661cc9d4b07022060990535eb51d164db2d5c68fd8528a0fc0e3ffa8cbb811a7a4527e572a0aa11, sighash=52292c9d613d8f7720d8c8d1506
cbb015fc65817f97ef3bb5def5e9f8f3b8475):
  result: success
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------
                                                                |                                                          01
btcdeb>
```

Figure 9: Step 7 (OP_CHECKSIG): ECDSA verification complete. sigversion=SIGVERSION_BASE. result: success. Final stack: [01] = TRUE

3.4 Final Result

OP_CHECKSIG result: success
sigversion = SIGVERSION_BASE (non-segwit / legacy style)
Final stack: `[01]` = TRUE
Transaction B to C is VALID

4 Part 2: P2SH-P2WPKH SegWit Transactions

4.1 Generated SegWit Addresses

Three P2SH-SegWit addresses were generated using `getnewaddress "label" "p2sh-segwit"`:

Label	Address
Address A'	2NB1Cu2NFNCcSznZKNoR9iMa4nbn9PpiJyz
Address B'	2NCnYAi5GMLZy5ZdKykoQjM6LfNXSuJE7yc
Address C'	2N595UZAVXjsRDH9XowziXSqDf9USSseyuo

4.2 Transaction Workflow: A' to B' to C'

Funding TXID (coins sent to A')

```
4919888a7cfb58e37cf80ee308edd94e8987f8b0393cb1e3257dc662090806bc
```

TXID: Transaction A' to B'

```
a450765063eaee4fc9a9ddc5af4f1ce1ec88255054f0187ff920719ae83fb872
```

TXID: Transaction B' to C'

```
c2531d0daa98f93eb7656946a4793fecd8c877d3b4f3b1cdfa079678339fa8f9
```

4.3 Decoded Scripts – Transaction A' to B'

Challenge Script (scriptPubKey) for Address B'

```
type: scripthash
asm:  OP_HASH160 d65675dc37e796a31825573b45d20467b48fc3cf OP_EQUAL
hex:  a914d65675dc37e796a31825573b45d20467b48fc3cf87
size: 23 bytes
```

This P2SH script commits only to the hash of the redeemScript. The actual redeemScript is revealed only when spending.

4.4 Decoded Scripts – Transaction B' to C'

Challenge Script (scriptPubKey) for Address C'

```
type: scripthash
asm:  OP_HASH160 8277e85f67e798b33ae34ce953f6ff84fe6d98a5 OP_EQUAL
hex:  a9148277e85f67e798b33ae34ce953f6ff84fe6d98a587
size: 23 bytes
```

Response Script (scriptSig) – spending B' UTXO

```
asm:  001404e5bad950f4ee912914c9d9eee8c173b7826102
hex:  16001404e5bad950f4ee912914c9d9eee8c173b7826102
size: 23 bytes only

Decoded redeemScript: OP_0 <04e5bad950f4ee912914c9d9eee8c173b7826102>
  = OP_0 <hash160(publicKey)>  <- P2WPKH witness program (22 bytes)
```

Key difference from P2PKH: scriptSig contains ONLY the 23-byte redeemScript. Signature and pubkey are segregated in the witness field.

Witness Data (segregated)

```
[0] signature (71 bytes, weight x1):
    304402203d1089a286d54166ad20b51377fb7a49ee298e529dfe5869d0f8ba30522bfa37
    022036b2c0f91a7f4c64c4659d4e2d3310e8783752a3fcedca71bea919ac7b5dc3fe01

[1] public key (33 bytes, weight x1):
    038019cda07cd3b7eb9421751ff0aac1b61a988d55973afa60fbe4293ab99c193f
```

Challenge-Response Match

Challenge (scriptPubKey)	Response (scriptSig + witness)
OP_HASH160 d65675dc... OP_EQUAL	scriptSig reveals redeemScript hash160(redeemScript) must == d65675dc... witness[1] pubkey hashed must match OP_0 program witness[0] signature verified against pubkey

4.5 Transaction Sizes – SegWit

Transaction	Size (bytes)	vSize (vbytes)	Weight (WU)
A' to B'	215	134	533
B' to C'	215	134	533

Raw size (215) is larger than P2PKH (191) because witness data is included. But vsize (134) is much smaller than P2PKH (191) due to the 75% witness discount.

5 Part 2: btcdeb Script Execution (SegWit)

5.1 Command Used

```
btcdeb --tx=<signed_hex_B_prime_to_C_prime>
       --txin=<signed_hex_A_prime_to_B_prime>
```

5.2 Initial Load – Witness Stack Size 2

```
(venv) sunny@DESKTOP-FVSQ52O:~/bitcoin_lab$ btcdeb --tx=020000000001018d402d205f02793fada73994741f8100cdde65144d9787177d29294b6406833a0000000017160014 0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0fdffffff01e092f505000
0000017a9140d3eb4ffbaff5652040bde767774bafd99bda30c870247304402201d9b606cfe9075032fa058261733ba571a728fc3f5726efb0e66dc50d2d323ec022076609928 5d538ad67ce3983c4a8248a57196269d9894afcf260949a6076fb53101210354ee007a
8066e07173496c9a7a7d8a001234237069bee07aac760b5dfd6d305400000000 --txin=020000000001010342f77739f06018326f4c5d44c9f769b95e582cb64b42888fe461db9ad115e900000000171600149764b5741244e873b9d45f415cf0e7e242e9cd45fdfff
fff01f0b9f5050000000017a9144ef689f589c6e76b6a7b656b1c2317ae02e7c8568702463043021f39deeba11059e81d2f3a6a9da21789178d1d6947fb6b6e143477ac3fcdd414022026c2c037962a3a0807f60764c5c3b98a1bb2fc5d5837fc34eb4af652cecdaffc
012102568c7dc5b8f7b32bb1a533f396d7692d872bb0b50812031b9dd64607711710830000000
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
input tx index = 0; tx input vout = 0; value = 99990000
got witness stack of size 2
script sig non-empty; embedded P2SH (extracting payload)
hash source = 00140d1ab23fbf56648d27e52b62e0b6a0965e24e6d0
22 bytes (P2WPKH)
valid script
- generating prevout hash from 1 ins
[+] COutPoint(3a8306644b, 0)
note: there is a for-clarity preamble (use --verbose for details)
5 op script loaded. type `help` for usage information
script                                   |                                                    stack
-----------------------------------------+----------------------------------------------------------
OP_DUP                                   | 0354ee007a8066e07173496c9a7a7d8a001234237069bee07aac760b5dfd6d3054
OP_HASH160                               | 304402201d9b606cfe9075032fa058261733ba571a728fc3f5726efb0e66dc5...
0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0 |
OP_EQUALVERIFY                           |
OP_CHECKSIG                              |
#0000 OP_DUP
btcdeb>
```

Figure 10: btcdeb: SegWit initial load – got witness stack of size 2, embedded P2SH detected, P2WPKH (22 bytes) extracted, 5 op script loaded. Stack pre-populated with pubKey and sig from witness.

The startup shows: `script sig non-empty; embedded P2SH (extracting payload)`, `22 bytes (P2WPKH)`, `5 op script loaded`. The inner P2WPKH script runs with the witness data already on the stack.

5.3 Step-by-Step Execution

```
btcdeb> step
                <> PUSH stack 0354ee007a8066e07173496c9a7a7d8a001234237069bee07aac760b5dfd6d3054
script                                   |                                                    stack
-----------------------------------------+----------------------------------------------------------
OP_HASH160                               | 0354ee007a8066e07173496c9a7a7d8a001234237069bee07aac760b5dfd6d3054
0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0 | 0354ee007a8066e07173496c9a7a7d8a001234237069bee07aac760b5dfd6d3054
OP_EQUALVERIFY                           | 304402201d9b606cfe9075032fa058261733ba571a728fc3f5726efb0e66dc5...
OP_CHECKSIG                              |
#0001 OP_HASH160
btcdeb>
```

Figure 11: Step 0 (OP_DUP): Public key `0354ee007a8066e07173496c...` duplicated. Stack: [pubKey, pubKey, sig]

```
btcdeb> step
                <> POP  stack
                <> PUSH stack 0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0
script                                   |                                                    stack
-----------------------------------------+----------------------------------------------------------
0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0 |                                0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0
OP_EQUALVERIFY                           | 0354ee007a8066e07173496c9a7a7d8a001234237069bee07aac760b5dfd6d3054
OP_CHECKSIG                              | 304402201d9b606cfe9075032fa058261733ba571a728fc3f5726efb0e66dc5...
#0002 0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0
btcdeb>
```

Figure 12: Step 1 (OP_HASH160): Top pubKey hashed – `0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0` computed and pushed

```
btcdeb> step
                <> PUSH stack 0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0
script                                  |                                                                   stack
----------------------------------------+-------------------------------------------------------------------
OP_EQUALVERIFY                          |                          0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0
OP_CHECKSIG                             |                          0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0
                                        | 0354ee007a8066e07173496c9a7a7d8a001234237069bee07aac760b5dfd6d3054
                                        | 304402201d9b606cfe9075032fa058261733ba571a728fc3f5726efb0e66dc5...

#0003 OP_EQUALVERIFY
btcdeb>
```

Figure 13: Step 2: Expected hash `0d1ab23fbf56648d27e52b62e0b6a0965e24e6d0` pushed for comparison. Both hashes now visible on stack.

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                  |                                                                   stack
----------------------------------------+-------------------------------------------------------------------
OP_CHECKSIG                             | 0354ee007a8066e07173496c9a7a7d8a001234237069bee07aac760b5dfd6d3054
                                        | 304402201d9b606cfe9075032fa058261733ba571a728fc3f5726efb0e66dc5...

#0004 OP_CHECKSIG
btcdeb>
```

Figure 14: Step 3 (OP_EQUALVERIFY): Both hashes match and verified – both popped. Stack: [pubKey, sig]

```
btcdeb> step
EvalChecksig() sigversion=1
Eval Checksig Pre-Tapscript
GenericTransactionSignatureChecker::CheckECDSASignature(71 len sig, 33 len pubkey, sigversion=1)
  sig         = 304402201d9b606cfe9075032fa058261733ba571a728fc3f5726efb0e66dc50d2d323ec0220766099285d538ad67ce3983c4a8248a57196269d9894afcf260949a6076fb53101
  pub key     = 0354ee007a8066e07173496c9a7a7d8a001234237069bee07aac760b5dfd6d3054
  script code = 76a9140d1ab23fbf56648d27e52b62e0b6a0965e24e6d088ac
  hash type   = 01 (SIGHASH_ALL)
SignatureHash(nIn=0, nHashType=01, amount=99990000)
- sigversion == SIGVERSION_WITNESS_V0
  sighash     = 24689457ac394cfd780cc42b4c73c01475bc57e36ff31acfd305de433759f223
  pubkey.VerifyECDSASignature(sig=304402201d9b606cfe9075032fa058261733ba571a728fc3f5726efb0e66dc50d2d323ec0220766099285d538ad67ce3983c4a8248a57196269d9894afcf260949a6076fb531, sighash=24689457ac394cfd780cc42b4c7
3c01475bc57e36ff31acfd305de433759f223):
  result: success
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                  |                                                                   stack
----------------------------------------+-------------------------------------------------------------------
                                        |                                                                     01

btcdeb>
```

Figure 15: Step 4 (OP_CHECKSIG): sigversion=SIGVERSION_WITNESS_V0 (BIP 143). ECDSA verification complete. result: success. Final stack: [01] = TRUE

5.4 Final Result

OP_CHECKSIG result: success
sigversion = SIGVERSION_WITNESS_V0 (BIP 143 sighash)
Final stack: `[01]` = TRUE
Transaction B' to C' is VALID

Critical difference from P2PKH: sigversion was `SIGVERSION_WITNESS_V0`, meaning BIP 143 sighash algorithm was used, which explicitly commits to the input value (prevents amount manipulation). The witness data was pre-loaded onto the stack before execution, not pushed via scriptSig.

6 Part 3: Comparative Analysis

6.1 Transaction Size Comparison

All values measured from actual transactions broadcast on regtest:

Metric	P2PKH (B-C)	SegWit (B'-C')	Difference	Savings
Size (bytes)	191	215	+24	SegWit larger raw
vSize (vbytes)	191	134	-57	29.8% smaller
Weight (WU)	764	533	-231	30.2% smaller

Raw byte size of SegWit is larger because it physically includes witness data. However, vsize is significantly smaller because witness bytes are counted at only 1/4 weight.

6.2 Script Structure Comparison

Component	P2PKH (Legacy)	P2SH-P2WPKH (SegWit)
scriptPubKey type	pubkeyhash	scripthash
scriptPubKey size	25 bytes	23 bytes
scriptPubKey opcodes	OP_DUP OP_HASH160 hash OP_EQUALVERIFY OP_CHECKSIG	OP_HASH160 hash OP_EQUAL
scriptSig content	signature + pubkey	redeemScript only
scriptSig size	106 bytes	23 bytes
Witness	none	signature + pubkey
Witness size	0 bytes	104 bytes
sigversion	SIGVERSION_BASE	SIGVERSION_WITNESS_V0
Sighash algorithm	Legacy double SHA256	BIP 143 (commits to value)

6.3 Weight Calculation – Worked Example

P2PKH (191 bytes, no witness)

```
All bytes are non-witness:
weight = 191 x 4 = 764 WU
vbytes = 764 / 4 = 191 vbytes
```

P2SH-P2WPKH (215 bytes total, 533 WU measured)

```
Non-witness bytes: ~108 bytes (version, inputs without sig, outputs, locktime)
Witness bytes:     ~107 bytes (sig ~71B + pubkey 33B + stack encoding ~3B)

weight = (108 x 4) + (107 x 1)
       = 432 + 101
       = 533 WU   (actual measured: 533)

vbytes = ceil(533 / 4) = 134 vbytes
```

6.4 Why SegWit Transactions Are Smaller

Reason 1: Witness Data Weight Discount

Bitcoin's weight unit system assigns different costs:

- **Non-witness data**: counted at weight **x4**
- **Witness data** (signatures, pubkeys): counted at weight **x1** (75% discount)

By moving signatures and pubkeys (104 bytes) to the witness field, their weight contribution drops by 75%.

Reason 2: scriptSig Weight Contribution

Part	P2PKH weight	SegWit weight
scriptSig (106 bytes vs 23 bytes)	106 x 4 = 424 WU	23 x 4 = 92 WU
witness (0 vs 104 bytes)	0 x 1 = 0 WU	104 x 1 = 104 WU
Total sig+pubkey contribution	**424 WU**	**196 WU**

Reason 3: Fee Benefit

```
Bitcoin fees = fee_rate (sat/vbyte) x vbytes

At 10 sat/vbyte:
  P2PKH:  191 vbytes x 10 = 1910 satoshis
  SegWit: 134 vbytes x 10 = 1340 satoshis
  Saving: 570 satoshis per transaction (~30%)
```

Reason 4: Block Throughput

Bitcoin block limit is 4,000,000 weight units. SegWit transactions use fewer weight units per transaction, allowing more transactions per block – increasing network throughput without a hard fork.

Reason 5: Transaction Malleability Fix

In P2PKH, signatures are inside scriptSig which is part of txid computation. A third party could alter the signature to produce a different txid – this is **transaction malleability**. In SegWit, signatures are in the witness field which is NOT part of txid, eliminating malleability and enabling the Lightning Network.

7 Conclusion

This lab successfully implemented Legacy P2PKH and SegWit P2SH-P2WPKH Bitcoin transactions on a local regtest network using Python and bitcoind RPC. All transactions were created, signed, broadcast, confirmed, and verified using btcdeb.

Key findings:

- The UTXO chain was correctly formed: txid `105e2c9e37...` (A to B) became input for B to C, and txid `a450765063...` (A' to B') became input for B' to C'.
- P2PKH challenge scripts: 25 bytes, `OP_DUP OP_HASH160 ... OP_EQUALVERIFY OP_CHECKSIG`. SegWit challenge scripts: 23 bytes, `OP_HASH160 ... OP_EQUAL`.
- P2PKH scriptSig: 106 bytes (signature + pubkey). SegWit scriptSig: only 23 bytes (redeemScript); 104 bytes move to witness.
- btcdeb confirmed `result: success` for both P2PKH (sigversion=BASE) and SegWit (sigversion=WITNESS_V0).
- Real measured savings: 191 vbytes (P2PKH) vs 134 vbytes (SegWit) = 29.8% reduction in effective size.
- SegWit BIP 143 sighash commits to input value, preventing amount manipulation attacks.

Repository: https://github.com/Sanskriti-hello/bitcoin-transaction-lab

Team HighOnByte | CS 216 | March 2026